

Goyer Golf MP Ladder

Refactor Plan — Speler/Account Architectuur

Project	Goyer Golf MP Ladder PWA
Versie	v2.5.63 → v3.0
Datum	16 april 2026
Status	Goedgekeurd voor implementatie

1. Probleemanalyse

De huidige architectuur kent drie overlappende databronnen die via fragiele naam-matching en ID-generatie gekoppeld zijn. Dit is de root cause van alle account/speler bugs.

Drie databronnen (huidig):

ladder/users	Accounts: email, uid, rol, playerId (koppeling via zelfgegenereerd playerId)
ladders/{id}.spelers[]	Ladder entries: naam, hcp, rank, partijen — duplicaat van spelerdata
ladder/alleSpelers	Master spelerslijst: naam, hcp — derde duplicaat

Geïdentificeerde bugs (gevolg van bovenstaande):

- getNextId() gebruikt lege in-memory cache bij registratie → geeft ID 1 terug → nieuwe gebruiker krijgt playerId van bestaande speler (bijv. Sierk Roosma)
- openEditPlayer zoekt op naam → vindt verkeerde account → toont email van andere speler
- store._ bezigMetRegistratie vs geïmporteerde primitive → race condition in onAuthStateChanged
- invite link check faalt zonder login door Firestore permission rules
- Drie kopieën van naam/hcp → kunnen uit sync raken

2. Ontwerpbeslissingen

De volgende keuzes zijn genomen na analyse en overleg:

Speler = Account	Elke speler in een ladder heeft een account. Geen spelers zonder account, geen accounts zonder speler (behalve tijdelijke gastspelers tijdens partij/toernooi).
Gastspelers	Bestaan alleen tijdens een partij of toernooi. Worden niet bewaard, komen nooit in een ladder.
Handicap	Één globale hcp per speler. Playing handicap per partij/toernooi wordt tijdelijk berekend op basis van baan en SI — nooit opgeslagen als ladder-hcp.
Naam wijzigen	Alleen door beheerder, om bijnamen te voorkomen.
Account verwijderen	Verwijderen uit ladder laat account intact. Beheerder verwijdert account apart als dat gewenst is.
Identificer	Firebase Auth uid is de enige identificer. Geen aparte spelerId, geen naam-matching, geen ID-generatie.

3. Nieuwe Datastructuur

Collectie: spelers/{uid}

Document-ID = Firebase Auth uid. Één bron van waarheid per speler.

```
{

  uid: "firebase-auth-uid", // = document ID

  naam: "Sierk Roosma",

  email: "sierk@gmail.com",

  rol: "beheerder", // speler | coordinator | beheerder

  hcp: 4

}
```

Collectie: ladders/{ladderId}

Bevat alleen metadata en een geordende lijst van uids. Geen duplicaat spelerdata.

```
{

  naam: "MP Ladder",
```

```
type: "regulier", // regulier | knockout | dames

config: { ... },

spelerIds: ["uid1", "uid2", "uid3"] // volgorde = ranking

}
```

Subcollectie: ladders/{ladderId}/standen/{uid}

Competitiedata per speler per ladder. Geen naam of hcp — die staan in spelers/{uid}.

```
{

  rank: 3,

  partijen: 12,

  gewonnen: 8

}
```

Wat verdwijnt:

- ladder/users document
- ladder/alleSpelers document
- ladders/{id}.spelers[] array
- getNextId() functie
- Alle naam-matching logica
- spelerId veld in users lijst

4. Firestore Security Rules

```
rules_version = '2';

service cloud.firestore {

  match /databases/{database}/documents {

    function isIngelogd() { return request.auth != null; }

    function isBeheerder() {
```

```

return
get(/databases/$(database)/documents/spelers/$(request.auth.uid))

.data.rol == 'beheerder';

}

function isCoordinator() {

let rol = get(/databases/$(database)/documents/spelers/$(request.
auth.uid)).data.rol;

return rol == 'coordinator' || rol == 'beheerder';

}

// Spelers – eigen profiel of beheerder

match /spelers/{uid} {

allow read: if isIngelogd();

allow write: if request.auth.uid == uid && onlyHcp()

|| isBeheerder();

}

// Ladders – lezen voor iedereen ingelogd, schrijven coordinator+

match /ladders/{ladderId} {

allow read: if isIngelogd();

allow write: if isCoordinator();

match /standen/{uid} {

allow read: if isIngelogd();

allow write: if isCoordinator();

}

}

```

```

}

// Toernooien – publiek leesbaar voor live meekijken

match /toernooien/{doc} {

  allow read: if true;

  allow write: if isIngelogd();

}

// Invite documenten – publiek leesbaar voor token validatie

match /ladder/{doc} {

  allow read: if isIngelogd() || doc.matches('invite_.*');

  allow write: if isIngelogd();

}

match /{document=**} {

  allow read, write: if false;

}

}

}

```

5. Registratie Flow (nieuw)

Stap 1	Beheerder genereert invite link voor een specifieke ladder
Stap 2	Nieuwe speler opent link — geen login vereist om de invite te valideren (public read op invite_* doc)
Stap 3	Speler vult naam, email, wachtwoord, hcp in
Stap 4	Firebase Auth account aangemaakt → uid beschikbaar

Stap 5	spelers/{uid} aangemaakt: naam, email, hcp, rol: "speler"
Stap 6	ladders/{ladderId}/standen/{uid} aangemaakt: rank (onderaan), partijen: 0, gewonnen: 0
Stap 7	ladders/{ladderId}.spelerIds bijgewerkt met uid
Stap 8	Succes — geen ID-generatie, geen naam-matching, geen race conditions

6. Impact per Module

auth.js	setIngeleid: één read op spelers/{uid} — geen zoeklogica. registreerSpeler: schrijft naar spelers/{uid} en standen/{uid}. getUsers()/saveUsers()/getNextId() vervallen.
admin.js	Speler toevoegen/bewerken/verwijderen volledig herschreven op uid-basis. Geen account/speler koppeling meer.
ladder.js	Laadt spelerIds uit ladder, haalt profielen op uit spelers/ collectie, standen uit subcollectie.
partij.js	Spelers ophalen via uid. Gastspelers blijven tijdelijk object zonder uid.
ronde.js	Minimale wijzigingen — spelerdata structuur gelijk.
toernooi.js	Spelers via uid uit spelers/ collectie.
knockout.js	Minimale wijzigingen.
store.js	huidigeBruiker bevat voortaan uid als primaire identifier.

7. Migratiestrategie

De migratie van bestaande data naar de nieuwe structuur gebeurt via een eenmalig script dat de beheerder uitvoert.

Stap 1	Lees bestaande users lijst (ladder/users) — bevat uid, email, naam, rol, spelerId
Stap 2	Lees master spelerslijst (ladder/alleSpelers) — bevat naam, hcp
Stap 3	Per user: koppel via naam aan spelerslijst voor hcp, schrijf spelers/{uid}
Stap 4	Lees alle ladders — per ladder.spelers[]: zoek uid via naam, schrijf standen/{uid}

Stap 5	Update ladders/{id}: vervang spelers[] door spelerIds[]
Stap 6	Verifieer output: toon koppelingsrapport — welke spelers geen uid hebben
Stap 7	Beheerder bevestigt — pas dan worden oude documenten als "gemigreerd" gemarkeerd (niet verwijderd)
Stap 8	Na 2 weken stabiel gebruik: oude documenten verwijderen

Let op: spelers die wel in de ladder staan maar geen account hebben (handmatig toegevoegd) krijgen na migratie een placeholder. Beheerder stuurt hen een invite link om een account aan te maken.

8. Bouwfasen

Fase	Naam	Inhoud	Modules
1	Migratiescript	Eenmalig script dat bestaande Firestore data naar nieuwe structuur kopieert. Output: koppelingsrapport voor beheerder.	auth.js (script)
2	auth.js refactor	Login, registratie, setIngelogd op nieuwe structuur. getUsers/saveUsers/getNextId verwijderen.	auth.js
3	admin.js refactor	Spelerbeheer volledig herschreven op uid. Toevoegen, bewerken, verwijderen, rolbeheer.	admin.js
4	ladder.js + store.js	Ladder laden vanuit nieuwe structuur. spelerIds + standen subcollectie.	ladder.js, store.js
5	partij + ronde + toernooi	Spelerdata lezen uit nieuwe collectie. Gastspelers ongewijzigd.	partij.js, ronde.js, toernooi.js
6	Security rules	Nieuwe Firestore rules deployen. Testen op testomgeving.	Firebase Console
7	Test & validatie	Volledig testen op test/ omgeving. Registratie, login, partij, toernooi, knockout.	Alle modules
8	Live + opruimen	Deployen naar live. Na 2 weken oude Firestore documenten verwijderen.	GitHub Pages

9. Risico's en mitigatie

Naam-mismatch bij migratie	Spelers zonder exact overeenkomende naam in users én ladder worden geflagged. Beheerder koppelt handmatig voor go-live.
Spelers zonder account	Worden geïdentificeerd in migratierapport. Beheerder stuurt invite link.
Downtime	Geen — migratie werkt naast bestaande structuur. Pas na bevestiging wordt de app omgeschakeld.
Data verlies	Oude documenten worden niet verwijderd, alleen gearhiveerd. Altijd terugrolbaar.
Context window	Refactor start in nieuwe conversatie met verse context, volledige codebase als zip, en dit plan als referentie.

10. Startpunt nieuwe conversatie

Stuur het volgende mee bij het starten van de nieuwe conversatie:

- De huidige zip met alle bestanden (root-v2.5.63.zip)
- Dit PDF document als referentie
- De werkregels (geen aannames, summary eerst, wacht op toestemming, zip per versie)
- Vermelding: nieuwe conversatie, start met migratiescript, bouw daarna in fasen zoals beschreven

Niet meesturen: de huidige auth.js bugs — die worden in de refactor volledig vervangen.